

“SignBridge – Intelligent Two-Way Sign Language Translator”

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

HASINI K 2520090209

LAVANYA L 2520090009

Under the Esteemed Guidance of

[Guide Name]

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University – Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)



Declaration

The Project Report entitled “**SignBridge – Intelligent Two-Way Sign Language Translator**” is a record of bonafide work of HASINI K – 2520090209, LAVANYA L – 2520090009, submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section [Section No.], Team 29, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

HASINI K – 2520090209

LAVANYA L – 2520090009

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**SignBridge – Intelligent Two-Way Sign Language Translator**” is a bonafide work done and submitted by HASINI K – 2520090209, LAVANYA L – 2520090009 in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section [Section No.], Team 29, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our Course Coordinator, Dr. [Name], and our guide, [Guide Name], Department of Computer Science and Information Technology, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means to complete this project-based lab.

We thank all the teaching and non-teaching staff members who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

HASINI K – 2520090209

LAVANYA L – 2520090009

Abstract

SignBridge is an AI-powered, two-way sign language translation platform designed to reduce the communication gap between sign language users and the wider hearing community, especially in situations where a human interpreter is unavailable. Most existing tools are one-directional: they convert signs to text but cannot translate spoken or written language back into sign form, and they rarely flag uncertain predictions, so a misread gesture is passed on as a confident but incorrect translation.

SignBridge addresses these gaps by converting hand gestures into text and speech, and by converting text or voice input into visual sign representations, in real time. Hand gestures are captured with OpenCV, hand landmarks are extracted with MediaPipe, and a TensorFlow / TensorFlow Lite model classifies each gesture. A confidence-scoring mechanism detects uncertain predictions and asks the user to repeat unclear gestures.

Smart Phrase Builder combines recognized signs into meaningful sentences, an “Emergency Mode” gives one-tap access to critical phrases, and a “Teach the AI” feature lets users add their own personalized gestures to the vocabulary.

The application uses a React.js frontend, a FastAPI backend, and a MySQL relational database accessed through the SQLAlchemy ORM. The database holds ten related tables covering roles, users, sessions, gestures, translations, translation–gesture sequences, confidence feedback, emergency phrases, user emergency favorites, and audit logs, with relationships enforced through primary and foreign keys. The platform is secured with JWT authentication and role-based access control (RBAC) and is designed for Docker-based deployment.

At the current stage the dashboard UI, the full MySQL schema with sample data, the JWT authentication endpoints, and a working prototype of the gesture-recognition pipeline have been implemented. SignBridge aims to be a reliable, adaptive and accessible communication solution rather than a conventional gesture-detection system.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement	Acknowledgement	4
	Abstract	Abstract	5
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users	8–9
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development Tools and Frameworks	10
3	Technology Stack	3.1 Frontend; 3.2 Backend; 3.3 AI and Vision; 3.4 Database; 3.5 Version Control; 3.6 Authentication and Security	11
4	System Architecture	4.1 High-Level Architecture; 4.2 Description of Each Layer; 4.3 Deployment Architecture	12–13
5	Design	5.1 Database Design Principles; 5.2 ER Diagram / Schema; 5.3 Design Decisions and Assumptions	14–15
6	Implementation	6.1 Module-wise Implementation; 6.2 Front-end Logic; 6.3 API Integration; 6.4 Authentication and Authorization	16
7	Features	7.1 List of Main Features; 7.2 How Each Feature Works	17–18

Index

S.No.	Chapters	Topics	Page No.
8	Testing	8.1 API Testing; 8.2 Integration Testing; 8.3 Tools Used; 8.4 Test Cases and Results	19–20
9	Deployment	9.1 Steps to Deploy; 9.2 Environment Configuration; 9.3 Hosting	21
10	Challenges & Limitations	10.1 Issues Faced; 10.2 Solutions Applied; 10.3 Limitations	22
11	Future Enhancements	11.1 Planned Features; 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary; 12.2 What Was Achieved; 12.3 Skills Learned	24
13	References	Documentation, tools and resources used	25
14	Appendices	Code snippets; setup instructions; credentials; API summary; screenshots	26–29

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Sign language is the primary mode of communication for many Deaf and hard-of-hearing people, yet most hearing people do not understand it. In healthcare, education, public services and everyday conversations, a trained interpreter is often the only bridge between the two groups, and interpreters are not always available. When they are not, sign language users face delays, misunderstandings or complete exclusion from the conversation.

Existing software tools attempt to help, but they are typically one-directional and treat recognition as a purely visual classification problem. They convert a sign into text, but do not convert spoken or written language back into signs, and they give no indication of how certain the model is about its answer. SignBridge was built to address these limitations with a single platform that translates in both directions, is aware of its own uncertainty, and can be personalized by its users.

1.2 Problem Statement

Communication barriers persist between sign language users and the wider hearing community, especially when a human interpreter is unavailable. The system must (a) recognize hand gestures in real time and present them as text and speech, (b) convert text or voice input into visual sign representations, (c) avoid passing on misread gestures as confident translations, and (d) give Deaf and hard-of-hearing users a fast, dedicated way to communicate critical or emergency information when interpreter or app support is limited.

1.3 Scope of the Project

The project covers the full pipeline from gesture capture to meaningful output: landmark extraction, gesture classification, confidence scoring, phrase building, reverse (text/voice-to-sign) translation, an emergency phrase mode, user-taught custom gestures, a sign library and dashboard, and a secure backend with role-based access, audit logging and a relational database. Support for multiple sign language standards, continuous sentence-level recognition, a native mobile application and live video-call integration are treated as future work.

CHAPTER 1: INTRODUCTION (CONTINUED)

1.4 Objectives

- **Two-way translation:** convert hand gestures to text/speech, and text or voice into visual sign representations, in real time.
- **Reliable recognition:** use a confidence-scoring mechanism to detect uncertain predictions and prompt the user to repeat unclear gestures.
- **Personalized vocabulary:** let users teach the AI new, personalized gestures through the “Teach the AI” feature.
- **Emergency communication:** provide an Emergency Mode with quick access to critical phrases when time matters most.
- **Secure, scalable platform:** build on a modular, service-oriented architecture with JWT authentication, RBAC and Docker-based deployment.

1.5 Importance of the Application

SignBridge moves beyond conventional gesture detection toward an inclusive, two-way communication platform. Sign language users gain a way to be understood without waiting for an interpreter, and hearing individuals unfamiliar with sign language gain a way to respond in a form the sign language user can read. Confidence-aware translation reduces the risk of silent errors, which matters most in high-stakes settings such as healthcare. The Emergency Mode and personalized vocabulary make the tool practical for real situations rather than a demonstration of recognition accuracy alone.

1.6 Target Users / Audience

The system serves sign language users (Deaf and hard-of-hearing individuals), hearing individuals who do not know sign language, and administrators who manage the platform. Access is governed by three roles: admin, user and guest.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- A camera-enabled device (webcam or phone camera) for real-time hand gesture capture.
- A processor and memory sufficient for computer vision inference; TensorFlow Lite optimization is used so the system can run on modest hardware.
- Audio output (speakers or headphones) for spoken translations, and a microphone for voice input.

2.2 Software Requirements

- Python (for FastAPI, OpenCV, MediaPipe and TensorFlow) and Node.js with npm (for the React frontend).
- MySQL 8.0 or later.
- A modern web browser with camera and microphone permissions (Chrome, Edge or Firefox).
- Docker, for containerized deployment.
- Git, for version control.
- MySQL Workbench and Visual Studio Code, for development (optional but recommended).

2.3 Development Tools and Frameworks

React.js was used for the frontend; FastAPI for the backend REST API; MySQL as the relational database, accessed through the SQLAlchemy ORM; OpenCV and MediaPipe for hand landmark detection; TensorFlow/Keras and TensorFlow Lite for gesture classification; Docker for deployment; and Git with GitHub for source control. VS Code was the primary development environment, and MySQL Workbench was used for inspecting and verifying the schema and data.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a React.js single-page application providing a responsive user interface. Its main screens are Dashboard/Home (quick-launch hero and live usage statistics), Translate (real-time gesture-to-text/speech conversion), Phrase Builder, Emergency Mode, Teach the AI, and Sign Library.

3.2 Backend

The backend is a REST API built with FastAPI, chosen for its high performance and automatic API documentation. It follows a modular, service-oriented structure in which each functional area (authentication, gestures, translation, phrases, emergency, audit) is kept in its own module.

3.3 AI and Vision Stack

OpenCV captures video frames and MediaPipe detects hand landmarks. A TensorFlow/Keras model, optimized with TensorFlow Lite for real-time inference on modest hardware, classifies the extracted landmark data into gesture labels.

3.4 Database

MySQL stores all persistent data across ten related tables: role, user, session, gesture, translation, translation_gesture, confidence_feedback, emergency_phrase, user_emergency_favorite and audit_log. The SQLAlchemy ORM is used for data access. Referential integrity is enforced with primary and foreign keys, and composite keys are used in the linking tables.

3.5 Version Control

Git was used for version control, with the project hosted on GitHub.

3.6 Authentication and Security Libraries

Users authenticate with JSON Web Tokens (JWT). Role-based access control (RBAC) restricts each endpoint to the roles allowed to call it (admin, user, guest). Sessions and sensitive actions are recorded in the session and audit_log tables.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

SignBridge follows a modular, service-oriented, layered architecture that separates the user interface, the business and AI logic, and the persisted data. Figure 4.1 shows the layers and how a request flows between them.

Layer	Main Responsibility
Presentation	React.js application; camera and microphone capture; translation, phrases, emergency options and sign library.
Application	FastAPI REST API; JWT verification; RBAC; authentication, gesture recognition, two-way translation, phrase building, confidence feedback, emergency phrases, Teach the AI, dashboard and sign library.
AI / Vision	OpenCV and MediaPipe landmark extraction; TensorFlow / TensorFlow Lite gesture classification; confidence score.
Data	MySQL accessed through SQLAlchemy; users, roles, sessions, gestures, translation history, feedback, emergency phrases and audit logs.

Figure 4.1: High-level architecture summary. The original supplied project documentation specifies an architecture diagram placeholder from Review 2, Slide 5; that source figure was not embedded in the supplied PDF.

4.2 Description of Each Layer

Presentation Layer. The React.js application captures the camera feed and microphone input, and displays translations, phrases, emergency options and the sign library. It communicates with the backend only through REST API calls.

Application Layer. The FastAPI backend receives requests, verifies the JWT, applies role-based access rules, and routes the request to the appropriate functional module.

AI / Vision Layer. OpenCV and MediaPipe extract hand landmarks from captured frames, and the TensorFlow / TensorFlow Lite model classifies the gesture and returns a label with a confidence score. Predictions below the confidence threshold trigger a repeat prompt instead of a translation.

Data Layer. The MySQL database, accessed through SQLAlchemy, stores users, roles, sessions, gestures, translation history, feedback, emergency phrases and audit logs.

CHAPTER 4: SYSTEM ARCHITECTURE (CONTINUED)

4.3 Deployment Architecture

The frontend, backend and database are designed as separate services deployed with Docker, so each can be built, started and scaled independently. During development, all services run locally on one machine. The source documentation leaves the exact frontend and FastAPI ports to be added; MySQL is identified as using port 3306.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema is normalized so that each fact is stored in exactly one place and relationships between entities are expressed through foreign keys rather than duplicated data. Many-to-many relationships (a translation containing several gestures; users saving several emergency phrases) are resolved through linking tables with composite primary keys. Keys and constraints are defined in the database so that invalid or orphaned data is rejected at the database level.

5.2 ER Diagram / Database Schema

The logical ER diagram (crow's foot notation) is shown in Figure 5.1, and the ten tables are summarized below. The supplied project documentation identifies the intended figure as the logical ER diagram from Review 2, Slide 8; the original figure itself was not embedded in the supplied PDF.

Table	Key Columns	Purpose
role	role_id (PK), role_name, description	Defines admin / user / guest roles
user	user_id (PK), full_name, email, role_id (FK)	Stores each account and its role
session	session_id (PK), user_id (FK), jwt_token, device_info	Tracks login sessions
gesture	gesture_id (PK), gesture_label, landmark_data, created_by (FK)	Known signs and user-taught custom gestures
translation	translation_id (PK), user_id (FK), session_id (FK), confidence_score	One logged translation event
translation_gesture	translation_id (PK, FK), gesture_id (PK, FK), sequence_order	Ordered gestures that make up a translation
confidence_feedback	feedback_id (PK), translation_id (FK), predicted_label, actual_label	User corrections of low-confidence predictions
emergency_phrase	emergency_id (PK), phrase_text, category	Predefined critical phrases
user_emergency_favorite	user_id (PK, FK), emergency_id (PK, FK), added_at	Each user's favorite emergency phrases
audit_log	log_id (PK), user_id (FK), action, timestamp	Record of user and admin actions

CHAPTER 5: DESIGN (CONTINUED)

5.2 ER Diagram / Database Schema — Relationships

Relationships: a role has many users; a user has many sessions, translations, custom gestures and audit-log entries; a session has many translations; a translation has many gestures (through translation_gesture) and many feedback records; users and emergency phrases are related many-to-many through user_emergency_favorite.

5.3 Design Decisions and Assumptions

- **Ordered gesture sequences:** the translation_gesture table carries a sequence_order column so the Smart Phrase Builder can rebuild a sentence in the order the signs were made, and so the same gesture can be reused across many translations.
- **Custom gestures share the gesture table:** user-taught gestures are stored in the same gesture table as default signs, distinguished by created_by, so recognition and the sign library treat both uniformly.
- **Feedback is stored separately:** confidence_feedback records predicted versus actual labels, creating data that can later be used to improve the model.
- **Role separation:** roles live in their own table so permissions can be extended without changing the user table.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

Module	Functionality
Authentication & RBAC	JWT-based login with role-based access control (admin / user / guest)
Gesture Recognition Engine	OpenCV + MediaPipe landmark extraction and TensorFlow/TF-Lite classification
Two-Way Translation Engine	Gesture → text/speech and text/voice → sign visual, in real time
Smart Phrase Builder	Combines a sequence of recognized signs into a coherent sentence
Confidence Feedback Loop	Flags low-confidence predictions and lets the user correct them
Emergency Mode	One-tap access to critical, predefined phrases
Teach the AI	Lets users add and label personalized custom gestures

The central logic is the recognition pipeline: capture landmarks, classify the gesture, score confidence, and either accept the prediction or prompt the user to repeat it, then pass accepted signs to the phrase builder to form a sentence.

6.2 Front-end Logic

The React frontend is organized around the six main screens described in Chapter 7. The dashboard shows live usage statistics (known signs, saved phrases, translations logged and custom gestures). Shared components are written once and reused across screens so loading, error and confirmation behavior is consistent. The source documentation leaves the exact state-management approach as a project detail to be added.

6.3 API Integration

The frontend calls the FastAPI REST endpoints over HTTP and attaches the user's JWT to every protected request. Request and response bodies are validated by the backend, and FastAPI's built-in documentation exposes every endpoint for inspection and testing.

6.4 Authentication and Authorization

Users log in and receive a signed JWT, which is checked on every protected endpoint. RBAC then restricts access by role: administrators can manage users, signs and phrases; registered users can translate, save phrases, teach custom gestures and use Emergency Mode; guests have limited access. Sessions are stored in the session table, and significant actions are written to `audit_log`.

CHAPTER 7: FEATURES

7.1 List of Main Features

User-facing features:

- Real-time gesture-to-text and gesture-to-speech translation
- Text or voice to visual sign translation (reverse translation)
- Smart Phrase Builder that combines signs into full sentences
- Teach the AI: lets users add and label personalized custom gestures
- Dashboard & Sign Library: tracks known signs, saved phrases, translations and custom gestures
- Confidence-aware recognition with repeat prompts for unclear gestures
- Emergency Mode with one-tap critical phrases
- Sign Library to browse all known signs
- Dashboard with live usage statistics

Administrator-facing features:

- Role and user management through RBAC
- Management of the sign library and emergency phrases
- Audit log of user and system actions

CHAPTER 7: FEATURES (CONTINUED)

7.2 How Each Feature Works

Gesture recognition and translation. The camera feed is processed frame by frame; OpenCV and MediaPipe extract hand landmarks, and the TensorFlow/TF-Lite model returns a gesture label with a confidence score. Accepted labels are shown as text and spoken aloud.

Confidence feedback. If a prediction's confidence is below the threshold, the system does not translate it. It asks the user to repeat the gesture, and the user can correct a wrong prediction; the predicted and actual labels are stored for later improvement.

Smart Phrase Builder. Recognized signs are collected in order (recorded through `sequence_order`) and combined into a grammatically coherent sentence for the recipient. Phrases can be saved for reuse.

Reverse translation. Typed text or spoken input is converted to text and matched to signs in the library, which are then shown as visual sign representations.

Emergency Mode. A list of predefined critical phrases, grouped by category, is available with one tap. Users can mark phrases as favorites for even faster access.

Teach the AI. A user records a new gesture, labels it, and the landmark data is stored in the gesture table with the user as creator, extending the vocabulary beyond the default sign set.

Screenshot placeholder
Dashboard / Home and Translate pages
Insert screenshots from the running SignBridge application here.

CHAPTER 8: TESTING

8.1 API Testing

The FastAPI backend exposes interactive API documentation, which is used together with Postman to exercise each endpoint, including authentication, role restrictions and error responses. The source documentation states that the number of endpoints tested should be added once the API is complete.

8.2 Integration Testing

Each backend module is verified against a real, running MySQL database rather than mocked data, so that foreign keys, composite keys and constraints are exercised. The gesture-recognition pipeline (OpenCV + MediaPipe + TensorFlow) has been integrated in a working prototype and is evaluated under different lighting conditions, backgrounds and users.

8.3 Tools Used for Testing

- FastAPI interactive documentation and Postman, for API testing
- MySQL client / Workbench, for verifying table contents and constraints
- Manual browser testing, for the React frontend and live camera behavior

8.4 Test Cases and Results

The supplied documentation lists the following planned core test cases and explicitly leaves the Actual Result column for completion after execution.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401, login rejected	[Pass / Fail]
2	Normal user calls an admin-only endpoint	403 Forbidden	[Pass / Fail]
3	Request without a JWT	401 Unauthorized	[Pass / Fail]
4	Recognize a known sign with a clear hand	Correct label returned with high confidence	[Pass / Fail]
5	Perform an unclear or partial gesture	Low confidence; user is prompted to repeat	[Pass / Fail]
6	Submit a correction for a low-confidence prediction	Row saved in confidence_feedback	[Pass / Fail]

CHAPTER 8: TESTING (CONTINUED)

#	Test Case	Expected Result	Actual Result
7	Build a phrase from several signs	Sentence produced in the correct sequence_order	[Pass / Fail]
8	Text or voice input of a known word	Matching sign visual displayed	[Pass / Fail]
9	Open Emergency Mode and select a phrase	Phrase shown and spoken immediately	[Pass / Fail]
10	Add a custom gesture through Teach the AI	New row in gesture with created_by set	[Pass / Fail]
11	Add the same favorite emergency phrase twice	Rejected by composite primary key	[Pass / Fail]
12	Recognition under dim lighting / varied background	Accuracy recorded; low confidence handled safely	[Pass / Fail]

Testing status note

The source SignBridge documentation describes these as planned core tests and does not supply completed Pass/Fail outcomes. It does state that the recognition pipeline is a working prototype evaluated under different lighting conditions, backgrounds and users.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

- Load the database schema and sample data into MySQL.
- Start the backend: install Python dependencies and run the FastAPI app with Uvicorn. [Add exact command and port.]
- Start the frontend: cd client, npm install, npm run dev. [Add port.]
- Alternatively, start all services together with Docker. [Add docker compose command.]
- Open the application in a browser, allow camera and microphone access, and log in.

9.2 Environment Configuration

Configuration is kept in environment variables or a .env file that is never committed to version control: the database connection (host, port, user, password, database name), the JWT secret and token expiry, and allowed frontend origins for CORS. A .env.example template with placeholder values is committed instead.

9.3 Hosting of Frontend and Backend

The system is designed for Docker-based deployment, with the frontend, backend and database as separate containers. For this submission it runs locally for development and demonstration. [Update if a hosted deployment is done.]

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Achieving reliable hand-landmark detection under varying lighting, backgrounds and skin tones.
- Balancing model size and inference speed for real-time performance on modest hardware.
- Designing a confidence-scoring threshold that catches genuine errors without over-prompting the user.
- Structuring the database to support ordered gesture sequences (Smart Phrase Builder) and user-taught gestures cleanly.
- Implementing secure JWT authentication and role-based access control consistently across all API endpoints.

10.2 Solutions Applied

- Landmark-based recognition with MediaPipe, rather than raw pixels, reduces sensitivity to background and lighting; testing is carried out under varied conditions.
- The model is optimized with TensorFlow Lite to keep inference fast on modest hardware.
- The confidence threshold is tuned by testing, and the feedback loop stores corrections so the threshold and model can be refined.
- A linking table with a `sequence_order` column handles ordered gesture sequences, and custom gestures reuse the existing gesture table.
- Authentication and role checks are applied through shared dependencies so every protected endpoint follows the same rules.

10.3 Current Limitations or Known Bugs

- The gesture vocabulary is currently small (37 known signs) and is limited to isolated signs rather than continuous signing.
- Only one sign language standard is supported at present.
- Accuracy depends on camera quality, lighting and hand visibility.
- The system has not yet been tested with the Deaf and hard-of-hearing community.
- The backend API is still being completed, and the system has not been deployed to a public cloud environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Expand the gesture vocabulary and add support for multiple sign language standards (for example, ASL and ISL).
- Move from isolated sign classification to continuous, sentence-level recognition.
- Richer user feedback loops and adaptive fine-tuning to fit individual signing styles.
- A mobile application for on-the-go translation.
- Real-time translation integrated into live video calls.
- User testing with the Deaf and hard-of-hearing community to refine usability and accuracy.

11.2 Possible Integrations or Optimizations

- Train on a larger and more diverse dataset to improve model accuracy.
- Edge deployment with TensorFlow Lite on mobile and embedded devices.
- Strengthened monitoring pipelines and accessibility compliance.
- An automated test suite (for example, pytest) to complement manual testing.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to reduce the communication barrier between sign language users and the hearing community by building a two-way, AI-powered translation platform. It combines gesture recognition, confidence-aware translation, personalized learning and emergency communication in one accessible application.

12.2 What Was Achieved

At the current stage, the React.js dashboard with live usage statistics has been implemented; the ten-table MySQL schema has been designed and implemented with sample data (37 known signs, 3 saved phrases, 5 translations logged and 2 custom gestures); the FastAPI backend is under way with JWT authentication endpoints; and the OpenCV + MediaPipe + TensorFlow recognition pipeline is integrated in a working prototype. The result is a modular, secure foundation that supports long-term maintenance and growth.

12.3 Skills Learned

Working on this project gave practical experience in computer vision with OpenCV and MediaPipe; training and optimizing models with TensorFlow and TensorFlow Lite; relational database design with MySQL and SQLAlchemy, including many-to-many relationships; building REST APIs with FastAPI; implementing JWT authentication and role-based access control; building a React frontend; designing for accessibility; and Docker-based deployment.

CHAPTER 13: REFERENCES

1. React: Official Documentation, <https://react.dev>
2. FastAPI: Official Documentation, <https://fastapi.tiangolo.com>
3. SQLAlchemy: Official Documentation, <https://docs.sqlalchemy.org>
4. MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
5. OpenCV: Official Documentation, <https://docs.opencv.org>
6. MediaPipe Hands: Google AI Edge Documentation, <https://ai.google.dev/edge/mediapipe>
7. TensorFlow and TensorFlow Lite: Official Documentation, <https://www.tensorflow.org>
8. Docker: Official Documentation, <https://docs.docker.com>
9. JSON Web Tokens: <https://jwt.io>
10. MDN Web Docs: JavaScript and Web APIs reference, <https://developer.mozilla.org>

The references above are reproduced from the supplied SignBridge project documentation. No additional external references have been added to the project content.

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

Illustrative schema for the ordered gesture-sequence linking table (replace with the actual DDL or model):

```
CREATE TABLE translation_gesture (  
  translation_id INT NOT NULL,  
  gesture_id     INT NOT NULL,  
  sequence_order INT NOT NULL,  
  PRIMARY KEY (translation_id, gesture_id),  
  FOREIGN KEY (translation_id) REFERENCES translation(translation_id),  
  FOREIGN KEY (gesture_id)     REFERENCES gesture(gesture_id)  
);
```

Illustrative confidence check (replace with the actual code):

```
CONFIDENCE_THRESHOLD = 0.80 # tuned by testing  
def accept_prediction(label, confidence):  
  if confidence < CONFIDENCE_THRESHOLD:  
    return {"status": "repeat", "message": "Please repeat the gesture"}  
  return {"status": "ok", "label": label, "confidence": confidence}
```

CHAPTER 14: APPENDICES (CONTINUED)

Appendix B: Installation and Setup Instructions

- Create the MySQL database and load the schema and sample data. [Add commands.]
- Backend: create a virtual environment, pip install -r requirements.txt, then start FastAPI with Uvicorn.
- Frontend: cd client && npm install && npm run dev.
- Open the application in a browser and allow camera and microphone access.

Appendix C: Default Login Credentials

Role	Email	Password
Admin	[admin email]	[password]
User	[user email]	[password]
Guest	–	–

The supplied documentation uses placeholders for credentials; no real credentials are included in this consolidated report.

CHAPTER 14: APPENDICES (CONTINUED)

Appendix D: API Reference Summary

Full request and response examples are available in the FastAPI interactive documentation. The supplied project documentation identifies the following resource groups for the API reference:

Resource Group	Purpose / Scope described in project documentation
Auth	Authentication and JWT-based access
Gestures	Known and user-taught gestures
Translations	Two-way translation and translation history
Phrases	Smart Phrase Builder and saved phrases
Emergency	Emergency phrases and user favorites
Dashboard	Usage statistics and dashboard data
Admin	Administrative management through RBAC

The supplied documentation does not provide a complete endpoint-by-endpoint request/response list, so no additional endpoints or parameters have been invented.

CHAPTER 14: APPENDICES (CONTINUED)

Appendix E: Screenshots

Add screenshots from the running application in this order:

- 1. Login page
- 2. Dashboard / Home with live usage statistics
- 3. Translate page (gesture to text/speech)
- 4. Phrase Builder
- 5. Emergency Mode
- 6. Teach the AI
- 7. Sign Library
- 8. Sample database records (MySQL Workbench)

Screenshot Placeholder
Login Page
Insert screenshot from the running application here.

Screenshot Placeholder
Dashboard / Home
Insert screenshot from the running application here.

Screenshot Placeholder
Translate
Insert screenshot from the running application here.

Screenshot Placeholder
Phrase Builder
Insert screenshot from the running application here.

CHAPTER 14: APPENDICES (CONTINUED)

Screenshot Placeholder
Emergency Mode
Insert screenshot from the running application / database here.

Screenshot Placeholder
Teach the AI
Insert screenshot from the running application / database here.

Screenshot Placeholder
Sign Library
Insert screenshot from the running application / database here.

Screenshot Placeholder
MySQL Workbench Sample Records
Insert screenshot from the running application / database here.

End of Project Report